

Trading Risk Engine Technical Specification

Overview

This document specifies a production-grade trading risk engine that sits between signal generation and order routing. Its purpose is to prevent unauthorized, erroneous, oversized, duplicative, or poor-quality trades from reaching the market by enforcing pre-trade controls, trading-state controls, and operational governance. The design aligns with the core control principles described in SEC Rule 15c3-5 and FINRA market access guidance, including pre-set risk limits, management of financial exposure, control over erroneous orders, restricted system access, immediate post-event review, and ongoing control review.[cite:18] [cite:16]

The engine is designed as an independent policy and enforcement layer. It receives order intent from upstream strategy components, evaluates the intent against a deterministic rule pipeline, issues a decision, and records the full reasoning path for auditability and operational review.[cite:18] [cite:21]

Objectives

The engine should satisfy six core objectives:

- Prevent new trades when capital, exposure, or system-health thresholds are breached. [cite:18] [cite:16]
- Enforce pre-trade rejection of erroneous or non-compliant orders before they reach execution venues.[cite:18] [cite:17]
- Distinguish between normal operation, degraded operation, and emergency stop states. [cite:18] [cite:29]
- Preserve an auditable record of every trading decision, override, and configuration change.[cite:16] [cite:21]
- Support human supervisory control through authenticated overrides, reduce-only modes, and kill-switch actions.[cite:18] [cite:20]
- Adapt trade permissions to market regime, liquidity, volatility, and event risk rather than relying on signal quality alone.[cite:20] [cite:23]

System Positioning

The risk engine is not part of the signal engine and should not be embedded solely inside the execution layer. It should act as a mandatory checkpoint between trading intent and market routing so that no order can bypass the policy decision path.[cite:18] [cite:16]

A reference flow is:

1. Strategy emits signal.

2. Order intent builder normalizes the signal into a proposed order.
3. Risk engine evaluates the order intent.
4. Decision is returned as allow, allow-reduced, delay, reject, or reduce-only exit.
5. Only approved intents are handed to routing and execution.
6. Execution feedback updates exposure, P&L, state, and audit logs.[cite:18] [cite:21]

Functional Architecture

Services

Service	Primary responsibility
Signal Intake	Accept normalized trade requests from strategies or discretionary tools
Order Intent Builder	Convert signal metadata into a fully specified proposed order
Pre-Trade Risk Engine	Evaluate hard limits, soft limits, and marketability checks
Exposure Service	Maintain real-time positions, pending orders, and portfolio exposure
Market Regime Service	Tag current session, volatility regime, event windows, and liquidity state
State Machine Service	Maintain system risk state and transition logic
Override/Admin Service	Authenticate privileged actions such as unlock, resize, or halt
Audit/Event Store	Persist immutable decision, event, and change logs
Execution Adapter	Submit only approved orders and return acknowledgements/fills
Monitoring/Alerting	Notify operators of breaches, rejections, disconnects, and state changes

This separation supports direct and exclusive control over core risk settings and makes it easier to review and test each control surface independently.[cite:18] [cite:16]

Suggested internal modules

Within the pre-trade risk engine, implement the following modules in order:

- Identity and authorization checks
- Trading-state checks
- Order validity checks
- Exposure and concentration checks
- Market-quality checks
- Frequency and duplication checks
- Dynamic sizing logic
- Decision explanation and reason-code generator
- Audit writer[cite:18] [cite:21]

Order Decision Model

Each order request should resolve to one of five outcomes:

Decision	Meaning	Typical action
ALLOW	Trade passes all checks	Route order unchanged
ALLOW_REDUCED	Trade is valid but size must be cut	Route adjusted order
DELAY	Trade is temporarily blocked by transient conditions	Hold and re-evaluate after timer or event clears
REJECT	Trade violates a hard rule	Do not route
EXIT_ONLY	New risk is blocked but risk-reducing orders are permitted	Allow closes/reductions only

Every decision should include machine-readable reason codes and human-readable explanations so the system can display why a trade was approved, reduced, delayed, or blocked.[cite:21] [cite:24]

Pre-Trade Rule Set

Hard limits

Hard limits should reject the order immediately if breached.

Rule	Description	Example default
Max daily realized loss	Blocks new entries once realized loss exceeds threshold	2% of account equity
Max intraday drawdown	Blocks trading when equity falls from intraday peak beyond threshold	3%
Max order notional	Rejects oversized tickets	\$25,000 per order
Max shares/contracts	Rejects orders over position-size cap	Strategy-specific
Max symbol exposure	Prevents overconcentration in one name	10% gross
Max sector/theme exposure	Prevents hidden clustering across related names	20% gross
Max gross exposure	Caps total long + short book	100% gross
Max net exposure	Caps directional bias	30% net
Open-order exposure cap	Counts live orders toward exposure	Included in gross and symbol caps
Fat-finger price collar	Rejects orders too far from reference price	1%-3% depending on instrument
Duplicate-order suppression	Rejects repeated same-side same-symbol intents within a window	30-120 seconds

Rule	Description	Example default
Unauthorized action block	Rejects changes from unapproved users/services	Immediate

Pre-set financial limits and controls for erroneous orders are central features of market-access risk management expectations.[cite:18] [cite:16] [cite:17]

Soft limits

Soft limits should not necessarily reject an order; they may instead reduce size, impose delay, or trigger supervisor review.

Rule	Description	Typical response
Spread threshold	Block or resize in wide-spread conditions	DELAY or ALLOW_REDUCED
Slippage estimate threshold	Avoid poor-quality fills	ALLOW_REDUCED or REJECT
Liquidity threshold	Limit orders when depth is weak	DELAY
Volatility threshold	Pause entries in unstable conditions	DELAY
Session quality filter	Different rules for open, midday, close, overnight	DELAY or ALLOW_REDUCED
Consecutive losses	Pause after a loss cluster	DELAY or EXIT_ONLY
Strategy degradation	Pause when expectancy, fill quality, or win rate degrades	EXIT_ONLY
Message-rate threshold	Slow or stop a runaway loop	DELAY or REJECT

These controls support the broader objective of preventing the system from operating unsafely under poor or abnormal market conditions.[cite:20] [cite:23] [cite:29]

Exposure Model

The engine should compute exposure from both filled positions and working orders. Exposure that ignores pending orders can understate real risk during fast signal bursts or partial-fill conditions.[cite:17] [cite:21]

Minimum exposure dimensions:

- Symbol gross exposure
- Symbol net exposure
- Strategy exposure
- Sector or theme exposure
- Asset-class exposure
- Portfolio gross exposure
- Portfolio net exposure

- Correlation bucket exposure
- Overnight carry exposure
- Options Greeks exposure where applicable[cite:21] [cite:23]

Exposure formulas

Suggested core measures:

- Gross exposure = sum of absolute position notionals plus working-order notionals.
- Net exposure = long notionals minus short notionals.
- Concentration ratio = symbol notional divided by portfolio gross exposure.
- Drawdown = current equity minus peak equity over the active session or evaluation window.

These metrics should be recalculated on every new order, acknowledgment, fill, cancel, reject, and price update that materially changes risk.[cite:21] [cite:29]

Market Regime Layer

A robust engine should incorporate a market-regime service that tags order requests with current session and market-quality context. Session-based and condition-based filtering helps avoid trading low-quality periods that can degrade fills and edge capture.[cite:20] [cite:23]

Recommended tags:

Tag	Values
Session	Pre-market, Open, Midday, Power Hour, After-hours
Volatility Band	Low, Normal, Elevated, Extreme
Liquidity Band	Thin, Normal, Deep
Event Window	None, Earnings, CPI, FOMC, Fed speaker, Halt recovery
Spread State	Tight, Acceptable, Wide, Extreme
Trend State	Trend, Range, Reversal, Unclear

A rule should be able to say, for example: allow full size during trend-confirmed open conditions, allow half size during midday normal conditions, and reject during elevated-volatility event windows.[cite:20] [cite:23]

Trading State Machine

States

State	Description	Entry behavior	Exit behavior
ACTIVE	Normal operations	Allow new entries and exits	Standard routing
COOLDOWN	Temporary pause after losses or instability	Block new entries	Allow exits only
SOFT_BLOCK	Risk elevated but not catastrophic	Block new entries	Allow reduce-only orders
BREACH	Hard rule exceeded	Reject new risk	Allow emergency flattening only
KILL_SWITCH	Manual or automated emergency stop	Cancel working orders and reject new orders	Allow privileged emergency exits only
PROBATION	Post-breach reactivation	Permit smaller size under tighter rules	Gradually restore normal limits

Transition examples

From	Trigger	To
ACTIVE	Daily loss limit hit	BREACH
ACTIVE	3 losses in 15 minutes	COOLDOWN
ACTIVE	Manual halt command	KILL_SWITCH
COOLDOWN	Timer expires and system health normalizes	PROBATION
PROBATION	Stable performance over threshold window	ACTIVE
PROBATION	New loss cluster or reject spike	COOLDOWN
BREACH	Supervisor-approved reset after review	PROBATION
Any state	Connectivity loss or runaway order loop	KILL_SWITCH

Explicit state management is critical because a mature control system must distinguish temporary pause, reduce-only behavior, and full emergency stop rather than relying on a single on/off flag.[cite:18][cite:29]

Kill Switch Design

The kill switch should be both manual and automatic.

Manual kill switch

Manual kill switch capabilities should include:

- Reject all new orders immediately
- Cancel all working orders immediately
- Freeze strategy scheduling and signal publication

- Notify all operators and write an immutable event record
- Require privileged credentials for reactivation[cite:20] [cite:18]

Automatic kill switch triggers

Recommended automatic triggers:

- Daily loss or drawdown hard breach
- Duplicate-order storm
- Message-rate runaway
- Exchange or broker disconnect combined with stale state
- Position mismatch between internal ledger and broker state
- Unbounded slippage or fill anomaly spike
- Manual supervisor trigger from dashboard[cite:20] [cite:29]

Override and Authorization Model

Direct and exclusive control over core risk settings is a recurring regulatory expectation for market-access controls.[cite:18] [cite:16]

Recommended roles:

Role	Permissions
Viewer	Read dashboards and logs only
Operator	Trigger halt, flatten, and acknowledge alerts
Supervisor	Approve resets, override cooldowns, modify certain thresholds
Risk Admin	Change core risk policies and access controls
System Service	Submit signals or route orders, but cannot rewrite policy without explicit privilege

All overrides should be time-bound, reason-coded, and logged with before/after values, user identity, and expiration.[cite:16] [cite:21]

API Design

A minimal service interface can include the following endpoints.

Risk evaluation endpoints

```
POST /risk/evaluate
POST /risk/batch-evaluate
GET /risk/state
GET /risk/exposure
GET /risk/limits
```

Control endpoints

```
POST /risk/kill-switch
POST /risk/release-kill-switch
POST /risk/flatten
POST /risk/reduce-only
POST /risk/reset-breach
POST /risk/override
```

Audit and review endpoints

```
GET /risk/events
GET /risk/decisions/{decision_id}
GET /risk/overrides
GET /risk/config-history
```

Example evaluation request

```
{
  "timestamp": "2026-04-18T14:00:00Z",
  "account_id": "A1",
  "strategy_id": "open-breakout-v2",
  "signal_id": "SIG-92411",
  "symbol": "NVDA",
  "side": "BUY",
  "instrument_type": "EQUITY",
  "quantity": 100,
  "order_type": "LIMIT",
  "limit_price": 915.25,
  "time_in_force": "DAY",
  "intent_tags": ["breakout", "open-session"],
  "source": "strategy-engine"
}
```

Example evaluation response

```
{
  "decision_id": "DEC-4429001",
  "decision": "ALLOW_REDUCED",
  "approved_quantity": 50,
  "state": "PROBATION",
  "reason_codes": ["SYMBOL_EXPOSURE_NEAR_LIMIT", "MIDDAY_SIZE_HAIRCUT"],
  "human_explanation": "Trade permitted at half size because symbol exposure is near",
  "expires_at": "2026-04-18T14:00:05Z"
}
```


Database Schema

A relational model is sufficient for most implementations.

Core tables

Table	Purpose
accounts	Trading account metadata
strategies	Strategy definitions and assignments
risk_policies	Versioned policy sets
risk_limits	Individual thresholds per policy
positions	Current live positions
working_orders	Open and pending orders
fills	Execution history
risk_decisions	One row per evaluated intent
risk_decision_reasons	Reason-code detail for each decision
risk_events	Breaches, halts, resets, mismatches, anomalies
overrides	Human-approved temporary exceptions
state_transitions	History of state changes
market_regimes	Time-bucketed market-condition tags
config_changes	Who changed what and when
users	Authenticated human users and service identities

Example table definitions

```
CREATE TABLE risk_policies (  
  policy_id UUID PRIMARY KEY,  
  policy_name TEXT NOT NULL,  
  version INTEGER NOT NULL,  
  status TEXT NOT NULL,  
  created_at TIMESTAMP NOT NULL,  
  created_by TEXT NOT NULL,  
  approved_by TEXT,  
  effective_at TIMESTAMP NOT NULL  
);  
  
CREATE TABLE risk_limits (  
  limit_id UUID PRIMARY KEY,  
  policy_id UUID NOT NULL REFERENCES risk_policies(policy_id),  
  scope_type TEXT NOT NULL,  
  scope_value TEXT NOT NULL,  
  limit_name TEXT NOT NULL,  
  comparator TEXT NOT NULL,
```

```

    threshold_numeric NUMERIC,
    threshold_text TEXT,
    action_on_breach TEXT NOT NULL,
    enabled BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE risk_decisions (
    decision_id UUID PRIMARY KEY,
    timestamp TIMESTAMP NOT NULL,
    account_id TEXT NOT NULL,
    strategy_id TEXT NOT NULL,
    signal_id TEXT,
    symbol TEXT NOT NULL,
    proposed_qty NUMERIC NOT NULL,
    approved_qty NUMERIC,
    proposed_notional NUMERIC,
    decision TEXT NOT NULL,
    state_at_decision TEXT NOT NULL,
    latency_ms INTEGER,
    explanation TEXT
);

```

Audit Logging Specification

Every evaluation should emit an immutable record with:

- request payload hash
- evaluated policy version
- state at evaluation time
- measured values for every triggered rule
- final decision and approved size
- user or service identity
- latency in milliseconds
- downstream execution ID if routed[cite:16][cite:21]

A mature implementation should also maintain config history so operators can reconstruct whether a loss event was caused by market conditions, strategy logic, or a threshold change. [cite:16][cite:18]

Monitoring and Alerting

At minimum, publish alerts for:

- breach events
- kill-switch triggers
- reject-rate spikes
- stale market data
- stale position state

- broker/exchange disconnects
- internal/external position mismatches
- unusual slippage clusters
- repeated override usage
- policy changes outside approved windows[cite:20] [cite:29]

Recommended dashboards:

- Current state and risk budget remaining
- Live exposure by symbol, sector, strategy, and account
- Decision stream with filterable reason codes
- Breach timeline and recovery actions
- Fill quality and slippage trends
- Override history and pending approvals[cite:21] [cite:24]

Testing Plan

The risk engine should be tested at four levels.

Unit tests

Validate every rule independently with boundary values and malformed inputs.

Simulation tests

Replay historical days with realistic fill assumptions to confirm that limits trigger when expected and do not over-block valid trades.

Fault-injection tests

Force disconnects, duplicate signals, stale prices, fill mismatches, and order storms to verify kill-switch and safe-fail behavior.[cite:20] [cite:29]

User-acceptance and supervisory tests

Validate human workflows for halt, reset, override, flatten, and post-event review.[cite:16] [cite:18]

Deployment Guidance

Implementation should favor fail-closed behavior. If the risk engine is unavailable, stale, or unable to compute exposure reliably, new risk should be blocked rather than allowed.[cite:18] [cite:16]

Recommended deployment characteristics:

- Stateless API nodes with shared durable state store

- Low-latency cache for exposure snapshots
- Message bus for decisions, fills, and alerts
- Redundant connectivity to market data and broker/exchange state
- Clock synchronization across services
- Immutable event log retention for review and certification support[cite:16] [cite:21]

Options-Specific Extension

For options trading, extend the exposure service and limits model with:

- delta, gamma, vega, theta, and optionally rho budgets
- expiry-bucket exposure
- underlying concentration through delta-equivalent exposure
- short-vol and short-gamma concentration limits
- assignment and exercise risk checks
- spread-leg completeness validation
- option liquidity and bid-ask width thresholds[cite:21] [cite:23]

Recommended MVP Build Sequence

1. Build state machine service.
2. Build exposure aggregator with positions plus working orders.
3. Implement hard-limit pre-trade checks.
4. Add kill switch and flatten controls.
5. Add market regime filters.
6. Add duplicate suppression and cooldown logic.
7. Add audit/event store.
8. Add override workflows and admin console.
9. Add dashboards and post-trade analytics.[cite:18] [cite:21] [cite:24]

Final Design Principle

A trading system becomes a real risk-controlled platform when it can explain, in deterministic and auditable terms, why it did not trade. Pre-trade controls, supervisory access restrictions, erroneous-order protection, and continuous review are the main design pillars that separate an alert bot from a production-grade risk engine.[cite:18] [cite:16]